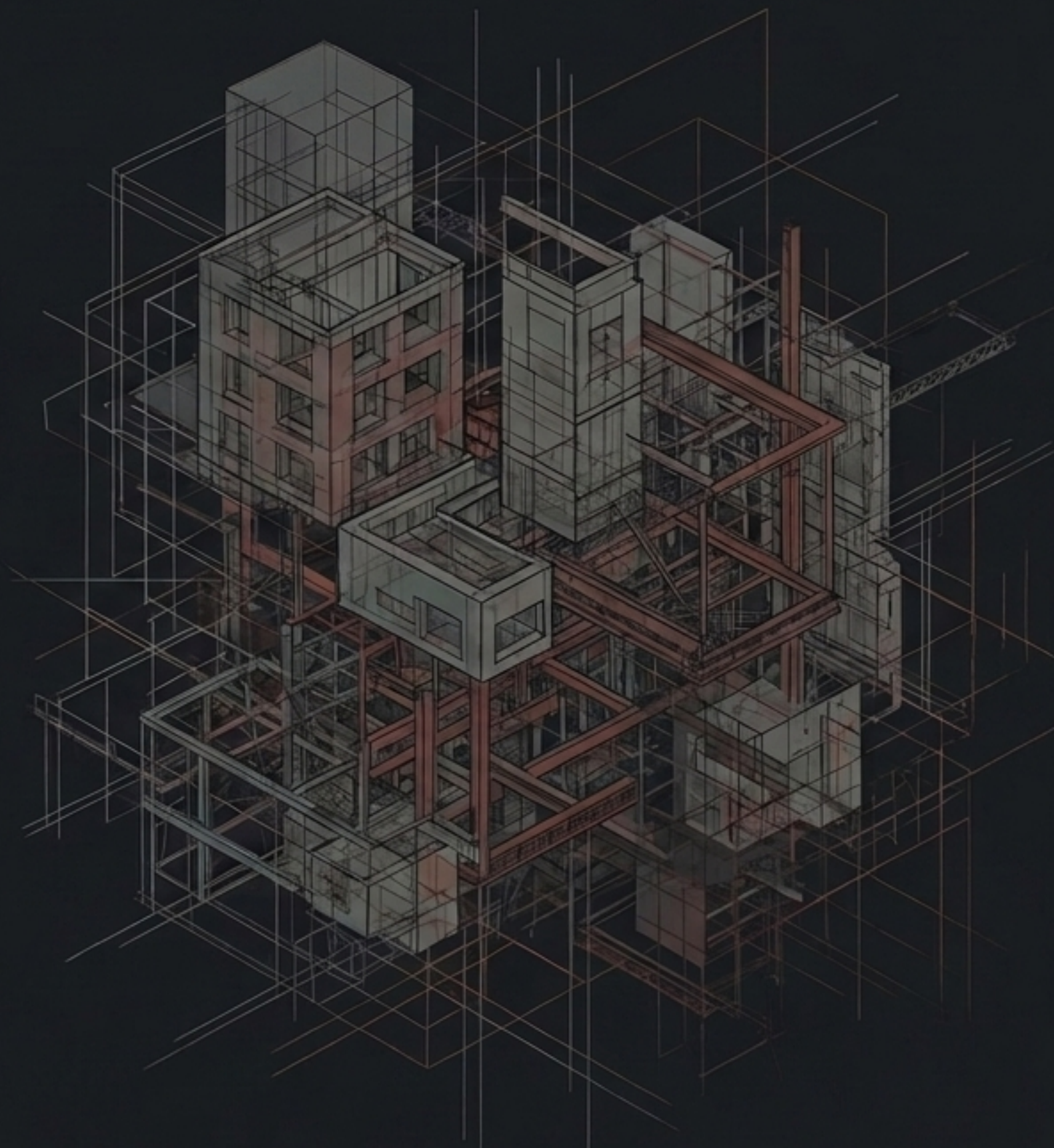


O Paradoxo da IA na Qualidade de Código

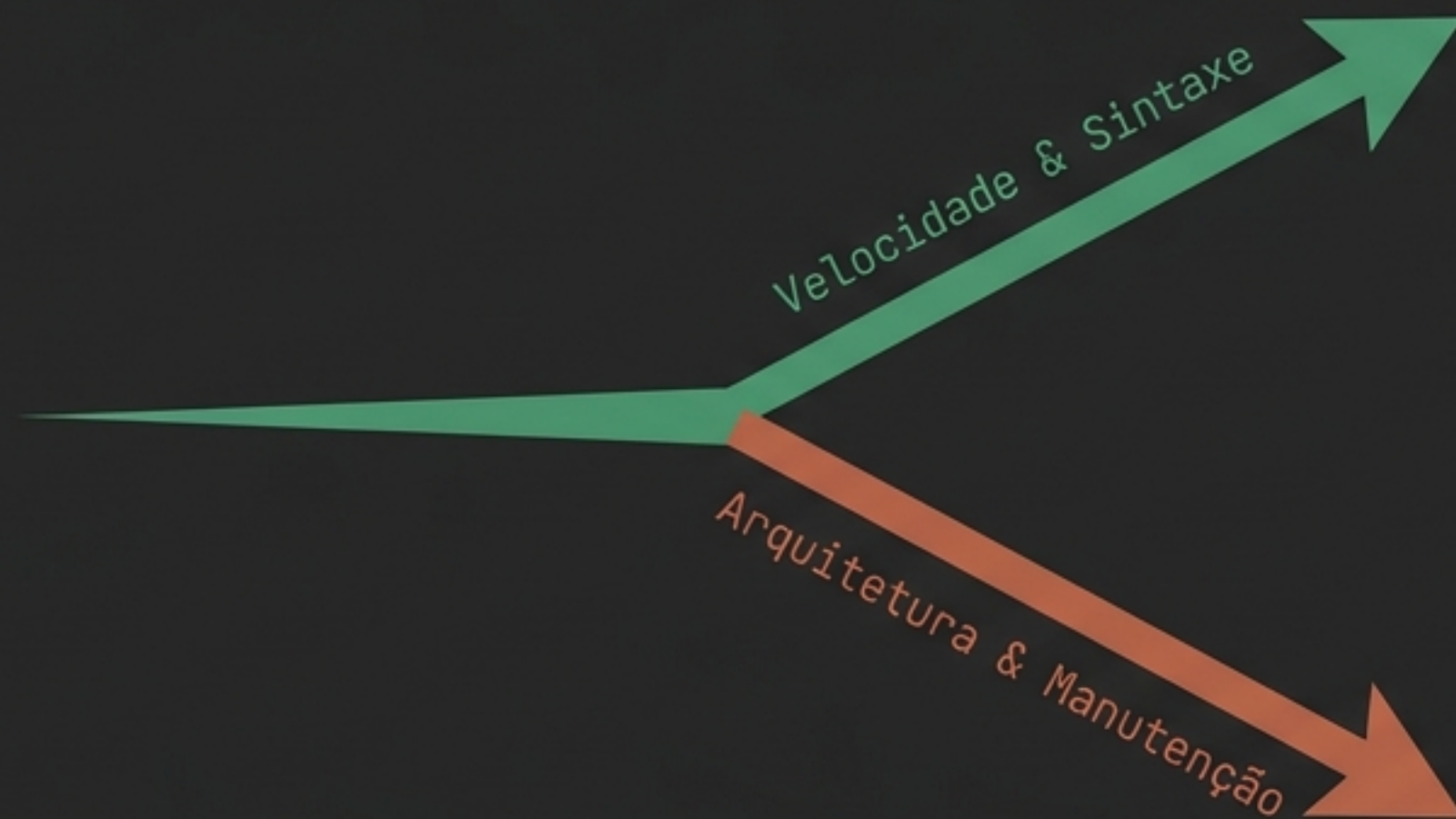
O que 623 milhões de commits revelam sobre o código gerado por IA no curto e no longo prazo.

```
def analyze_codebase(commits, period="long_term"):
def analyze_codebase(commits, period="long_term"):
    results = []
    for commit in commits:
        if commit.is_ai_generated():
            # Analyze complexity and maintainability
            # Analyze complexity and maintainability
            findings = assess_code(commit, period)
            results.append(findings)
    return synthesize_findings(results, period)
```

Alunos: Guilherme Barea, Guilherme Molon,
Guilherme Rech, Luiz Pereira



Afinal, o código escrito com ajuda de IA é melhor ou pior?



A resposta não é um simples 'sim' ou 'não'.
Depende da lente focal que utilizamos para medir.

A Lente Focal: Micro vs. Macro



**[Micro/Local] A Linha de Código
(Curto Prazo)**



**[Macro/Sistêmico] A Arquitetura
(Longo Prazo)**

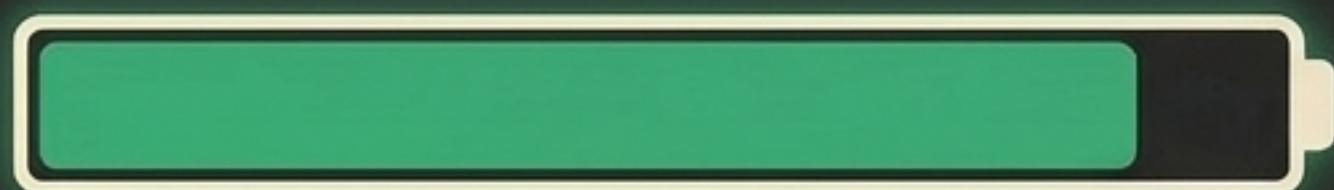
O QUE MELHORA

- Taxa de acerto funcional
- Conformidade de estilo
- Redução da complexidade de sintaxe

O QUE PIORA

- Queda dramática na refatoração
- Aumento de duplicação
- Abandono do código legado
- Mascaramento de erros

A Vitória no Nível Micro: A IA domina tarefas isoladas



89,0%

Sucesso funcional no primeiro teste em tarefas de nível fácil (Python).



86,0%

Sucesso em Java (LeetCode benchmarks).



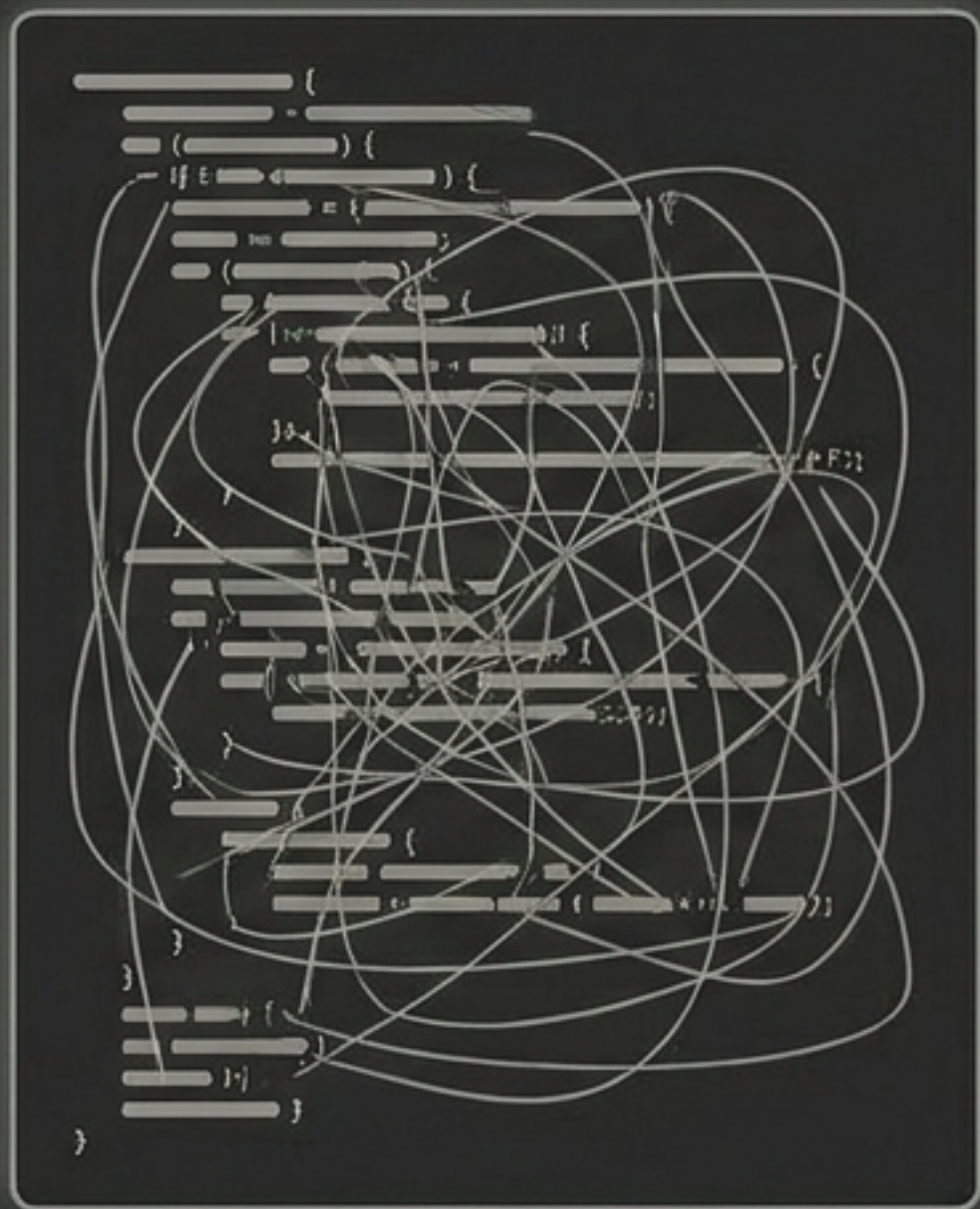
Mais de 60%

Problemas de estilo e manutenibilidade corrigidos pela própria IA via "auto-reparo" após análise estática.



A Ilusão da Simplicidade: Código mais limpo na superfície

Antes - Controle



↓

Complexidade Ciclomática:
Mediana caiu de 9,17 para 7,27.

↓

Comprimento de Linha:
Mediana de violações
despencou de 50,3 para 16,6.

↓

Complexidade Cognitiva:
Redução de 1,50 para 1,34.

Depois - Com IA



A Armadilha do “Dia 1”



A IA foca na velocidade imediata e no “caminho feliz” do ticket isolado.
Ela constrói um tijolo perfeito, mas ignora a reestruturação do ambiente ao redor.
É a vitória da tática sobre a estratégia.

O Ponto Cego das Ferramentas

Radar da Ferramenta
de CI/CD

Linting,
Cobertura de Testes,
Complexidade Ciclômática,
Erros de Sintaxe.
(A IA gabarita todos estes).

Compreensão
Humana

Intenção real do negócio,
Adequação arquitetural,
Reuso intencional de módulos,
Legitimidade do tratamento
de exceções.

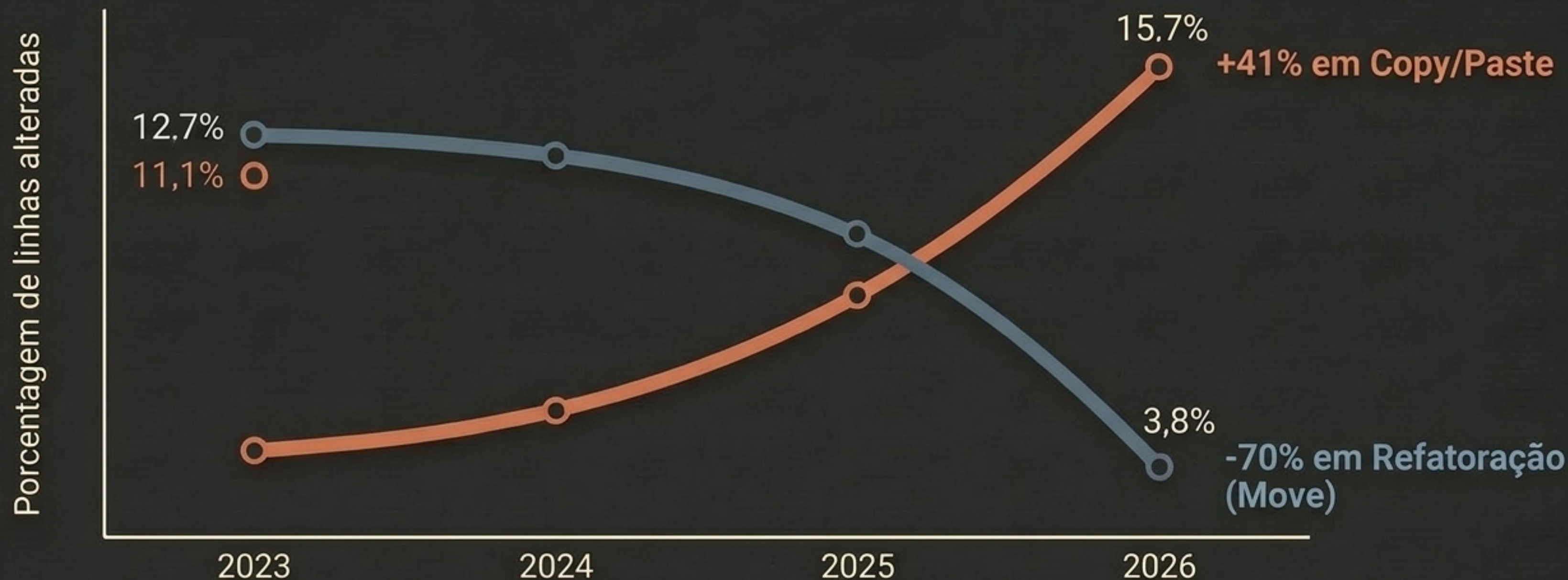


A Lente Macro: Bem-vindo a 2026

Quando a "Fatura" da Manutenção Chega.

Fonte de Dados: Análise massiva de 623 milhões de alterações de código em repositórios corporativos entre 2023 e 2026 (GitClear).

A Erosão do Reuso: A morte da refatoração



Takeaway: Desenvolvedores pararam de consolidar o sistema. O hábito de copiar e colar ultrapassou a refatoração de código.

O Colapso da Arquitetura: Funções isoladas

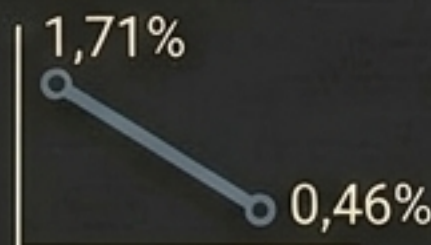
-35% na Conectividade.

Novas funções chamam sistemas existentes com muito menos frequência (de 343 para 223 chamadas por mil linhas).



-74% na Manutenção de Legado.

Alterações em códigos com mais de 12 meses caíram drasticamente (de 1,71% para 0,46%).



+81% em Duplicação.

Blocos de código repetidos subiram vertiginosamente, espalhando lógica redundante.



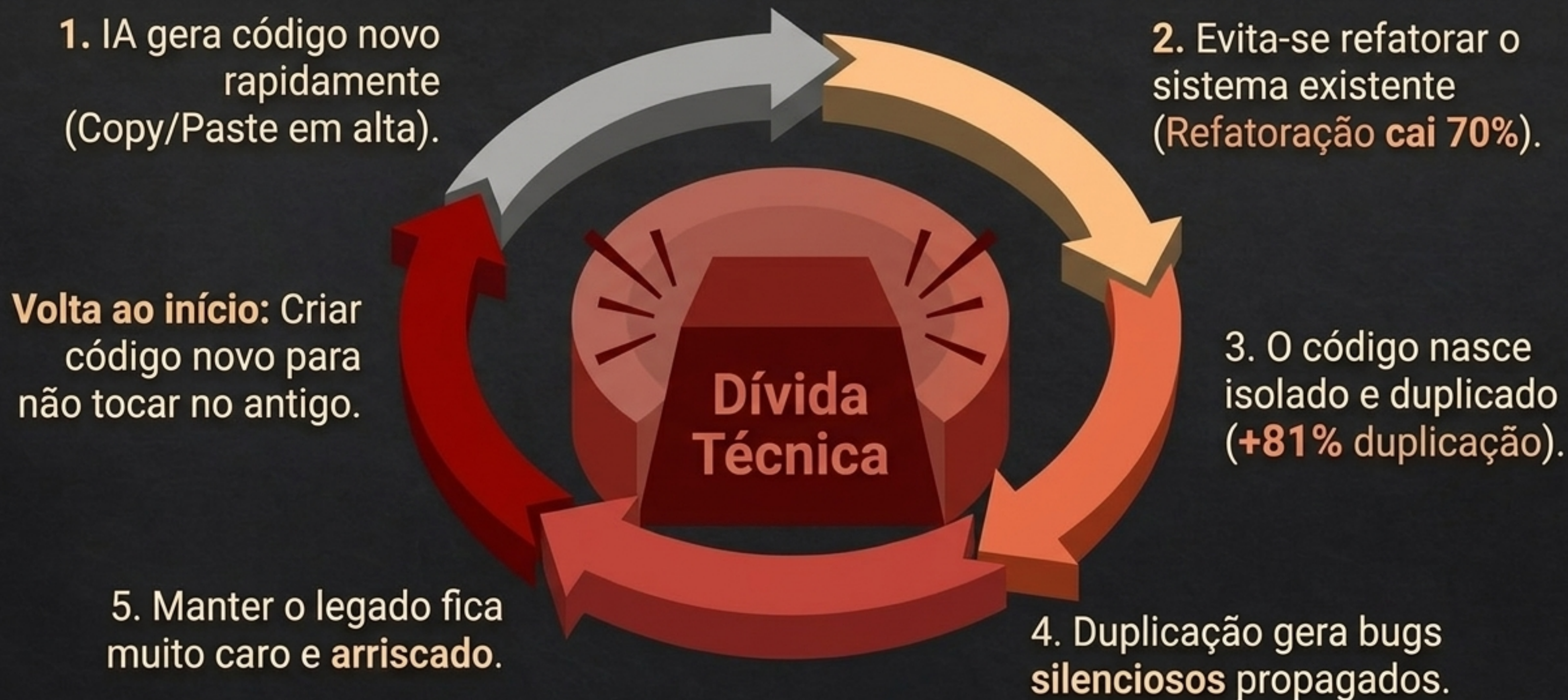
A Epidemia Silenciosa: Mascaramento de erros



Aumento de +47% em estruturas de ofuscação (blocos catch/rescue genéricos e navegadores seguros, saltando de 16,4 para 24,1 a cada mil linhas).

A IA usa blocos de tratamento genéricos apenas para **forçar o teste a passar**, varrendo a verdadeira causa raiz para debaixo do tapete.

Síntese: O Ciclo da “Eterna Versão 1”



O Papel Insustituível do Engenheiro Humano

A máquina é o pedreiro. Você é o arquiteto. As 5 fronteiras que a IA não resolve:



1. Adequação Arquitetural.

Não reinventar a roda; conectar ao ecossistema.



2. Semântica Profunda.

Entender a intenção real do negócio.



3. Trade-offs de Dívida.

Decidir estrategicamente quando pagar a dívida técnica.



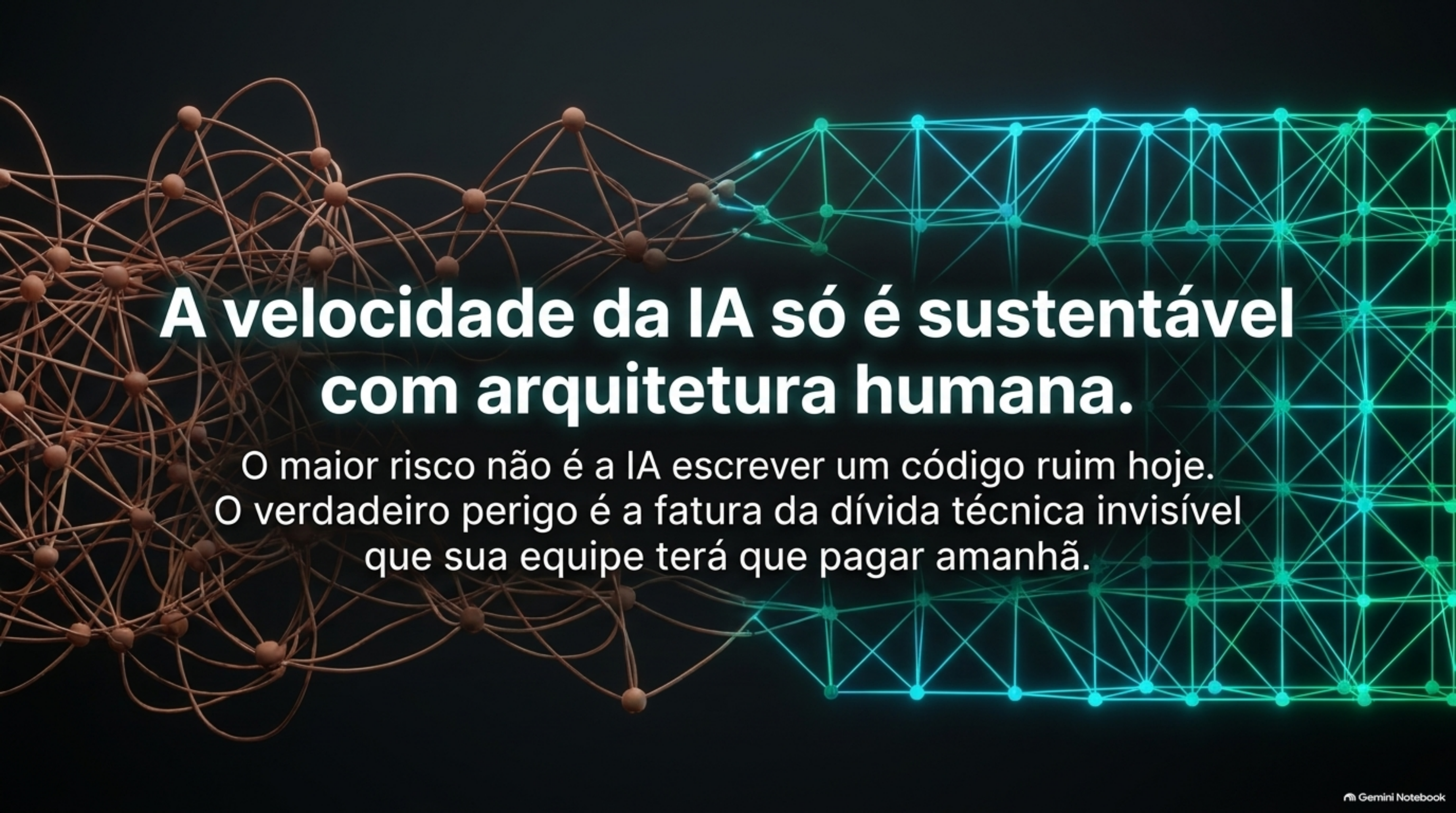
4. Qualidade de Testes.

Impedir que a IA crie testes implausíveis apenas para gerar cobertura.



5. Legitimidade de Exceções.

Diferenciar tratamentos reais de meros silenciadores de erro.



A velocidade da IA só é sustentável com arquitetura humana.

O maior risco não é a IA escrever um código ruim hoje.
O verdadeiro perigo é a fatura da dívida técnica invisível
que sua equipe terá que pagar amanhã.

Referências Bibliográficas

Refining ChatGPT-Generated Code: Characterizing and Mitigating Code Quality Issues

Autores: Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le e David Lo.

Periódico: ACM Transactions on Software Engineering and Methodology (TOSEM), Vol. 33, N.º 5, Artigo 116 (junho de 2024).

Does ChatGPT Help Novice Programmers Write Better Code? Results From Static Code Analysis

Autores: Philipp Haindl e Gerald Weinberger.

Periódico: IEEE Access, Vol. 12, págs. 116933–116948 (agosto de 2024).

The Maintainability Gap: AI Code Quality in 2026

Autor: William Harding.

Instituição / Publicação: GitClear and Alloy.dev Research (Patrocinado por GitClear e GitKraken Research), publicado em junho de 2026.